

Finding Eigenvalues with the QR Algorithm

ES.1806 Final Project

Chris Liu August 05, 2026

Assignment Overview

Document Contents:

1 Introduction 1

 1.1 Similar matrices 1

 1.2 Finding a single eigenvalue 1

2 The QR algorithm 2

 2.1 Orthogonal iteration 2

 2.2 Recurrence magic 3

 2.3 Example 4

3 Improving the QR algorithm 5

4 References 5

Assignment Details:

- **Course:** ES.1806 Linear Algebra
- **Instructor:** Arthur Parzygnat

Introduction

Eigenvalues and eigenvectors are an immensely useful aspect of linear algebra in that they have wide-ranging applications such as solving differential equations and finding high powers of matrices with little computation. However, the naive approach of finding eigenvalues, namely, solving the equation $\det(A - \lambda \mathbf{1}) = 0$ is computationally expensive and analytically impossible for polynomials of degree 5 or higher. In this project, we investigate an alternative method of computing eigenvalues and eigenvectors numerically: the QR algorithm. As the name suggests, the algorithm involves repeatedly finding the QR decomposition of a matrix.

Before we describe the QR algorithm, we need to explain two simple concepts, both of which are extensively used in the derivation of the QR algorithm.

1.1 Similar matrices

Two square $n \times n$ matrices A and B are similar if there exists an invertible matrix X such that $A = XBX^{-1}$.

Lemma 1 If A and B are similar, they have the same set of eigenvalues.

Proof Suppose λ_A is an eigenvalue of A , then by definition there exists $\mathbf{v}_A$ such that $A\mathbf{v}_A = \lambda_A\mathbf{v}_A$. Plugging in $A = XBX^{-1}$, we get

$$\begin{aligned} XBX^{-1}\mathbf{v}_A &= \lambda_A\mathbf{v}_A \\ \Rightarrow BX^{-1}\mathbf{v}_A &= \lambda_AX^{-1}\mathbf{v}_A \end{aligned} \tag{1}$$

Therefore, λ_A is an eigenvalue of B with eigenvector $X^{-1}\mathbf{v}_A$. If we replace X with its inverse, we can make an identical argument to prove that every eigenvalue of B is also an eigenvalue of A . $\square$

1.2 Finding a single eigenvalue

While it is difficult to find *all* eigenvalues of an arbitrarily large matrix, there is a straightforward algorithm to find the eigenvalue with the largest magnitude. For the sake of simplicity, we assume the $n \times n$ matrix A has *real, positive, distinct* eigenvalues. We also assume the eigenvalues $\lambda_1, \lambda_2, \dots, \lambda_n$ are arranged in strictly decreasing order, and that their corresponding eigenvectors are $\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_n$. Suppose the eigenvalue decomposition $A = X\Lambda X^{-1}$ also follows this order.

Consider a generic $n \times 1$ vector $\mathbf{x}$. We can rewrite the quantity $A^k\mathbf{x}$ as follows:

$$A^k\mathbf{x} = (X\Lambda X^{-1})^k\mathbf{x} = X\Lambda^k X^{-1}\mathbf{x} \tag{2}$$

What does this expression mean? $X^{-1}\mathbf{x}$ is the coordinates of $\mathbf{x}$ in the eigenbasis, so $X\Lambda^k X^{-1}\mathbf{x}$ would be a linear combination of the eigenvectors where the coefficient of each $\mathbf{v}_i$ is given by the product of λ_i^k and the i th coordinate of $\mathbf{x}$ (in the eigenbasis). To put it more clearly, suppose

$$\mathbf{x} = \sum_{i=1}^n c_i \mathbf{v}_i = X \begin{bmatrix} c_1 \\ \vdots \\ c_n \end{bmatrix}, \tag{3}$$

then

$$A^k \mathbf{x} = X \Lambda^k X^{-1} X \begin{bmatrix} c_1 \\ \vdots \\ c_n \end{bmatrix} = X \begin{bmatrix} \lambda_1^k & & \\ & \ddots & \\ & & \lambda_n^k \end{bmatrix} \begin{bmatrix} c_1 \\ \vdots \\ c_n \end{bmatrix} = X \begin{bmatrix} \lambda_1^k c_1 \\ \vdots \\ \lambda_n^k c_n \end{bmatrix} = \sum_{i=1}^n \lambda_i^k c_i \mathbf{v}_i \quad (4)$$

We can rescale the resulting vector by $\frac{1}{\lambda_1^k}$, where λ_1 is the largest eigenvalue:

$$\frac{1}{\lambda_1^k} A^k \mathbf{x} = c_1 \mathbf{v}_1 + \sum_{i=2}^n \left(\frac{\lambda_i}{\lambda_1} \right)^k c_i \mathbf{v}_i \quad (5)$$

because $\frac{\lambda_i}{\lambda_1} < 1$ for all $i \geq 2$, the second term disappears as we take the limit $k \rightarrow \infty$. The only remaining term is $c_1 \mathbf{v}_1$, and we have now found an eigenvector of A !

To summarize, the following algorithm finds the eigenvector with the largest eigenvalue for an $n \times n$ matrix A :

1. Start with any $n \times 1$ vector $\mathbf{x}$.
2. Multiply $\mathbf{x}$ by A .
3. Normalize the vector and go to step 2. (We technically don't need the normalization step, but it is difficult to determine convergence if the magnitude of the vector keeps changing.)

After an adequate number of iterations, the resulting vector will be a decent approximation of $\mathbf{v}_1$ and we can find the corresponding eigenvalue by using $A\mathbf{v}_1 = \lambda_1 \mathbf{v}_1$.

The QR algorithm

We motivate the QR algorithm by first looking at orthogonal iteration, which is an extension of the algorithm in Section 1.2. We then discuss how the canonical QR algorithm is merely a clever manipulation of the orthogonal iteration algorithm using recurrence relationships.

2.1 Orthogonal iteration

The basic idea of orthogonal iteration is that instead of finding a single eigenvector of A , we find the subspace spanned by the eigenvectors, i.e. $\mathbf{C}(X)$. If we maintain the same assumptions as Section 1.2 (real, positive, distinct eigenvalues), then A is diagonalizable, and we have $\mathbf{C}(X) = \mathbf{C}(A)$. In other words, the eigenvectors span the column space of A .

To actually implement the algorithm, we need to keep track of a matrix whose columns serve as the basis of the vector space we're trying to find. Specifically, we maintain an orthogonal matrix P_k (it is **not** called Q_k for a reason that will become evident in Section 2.2). The subscript k refers to the k th iteration of the algorithm.

The algorithm is as follows:

1. Choose any $n \times n$ matrix P_0 . Usually we pick $P_0 = \mathbb{1}_n$ for the purpose of simplicity.
2. Calculate AP_k .
3. Decompose the result into its QR factorization $P_{k+1}R_{k+1}$. (Once again, I apologize for the fact that the orthogonal matrix is not called Q .) Go to step 2.

Mathematically, the following recurrence relationship is maintained:

$$P_{k+1}R_{k+1} = AP_k \quad (6)$$

Notice that the same principles apply as in section Section 1.2. We repeatedly apply A to an entity and then rescale it in some way after each iteration, in this case using the QR factorization. In fact, if we only consider the first column $\mathbf{p}$ of matrix P on both sides of Equation 6, we have:

$$\mathbf{p}_{k+1} r_{k+1} = A \mathbf{p}_k \quad (7)$$

(r_{k+1} is a scalar because the matrix R_{k+1} is upper triangular.) If we squint our eyes at Equation 7, we will notice that it is precisely the same algorithm as Section 1.2: repeatedly apply A to a vector, and then normalize it!

A partial proof of correctness of the orthogonal iteration algorithm is given as follows:

Theorem 2 If the orthogonal iteration algorithm converges, the diagonal entries of R_k contain the eigenvalues of A .

Proof Because P_{k+1} is orthogonal, we can rewrite Equation 6 as:

$$R_{k+1} = P_{k+1}^{-1} A P_k = P_{k+1}^T A P_k \quad (8)$$

Convergence gives $R_{k+1} \approx R_k$, and $P_{k+1} \approx P_k$, which gives

$$R_k \approx P_k^T A P_k \quad (9)$$

In other words, R_k is similar to A . Lemma 1 shows that R_k has the same eigenvalues as A . Since R_k is upper triangular, its eigenvalues are simply its diagonal entries, because the determinant of an upper triangular matrix is equal to the product of its diagonals. $\square$

Proving the convergence of the orthogonal iteration algorithm is beyond the scope of this project.

2.2 Recurrence magic

Notice that in the orthogonal iteration algorithm, the orthogonal matrix P_k is what gets passed around between two consecutive iterations. However, P_k does not necessarily contain any useful information other than the fact that it is orthogonal and that it spans the column space of A . Specifically, it does **not** contain the eigenvectors of A .

In order to find the eigenvalues of A , we need to extract them from R_k , which is *only* similar to A *at convergence*. The canonical version of the QR algorithm mitigates this inconvenience by translating the orthogonal iteration algorithm into a recurrence relationship that produces a matrix which is *always* similar to A . As a result, we can keep executing this algorithm and directly read off the answer once the matrix has converged.

Define the matrix

$$A_k = P_k^T A P_k \quad (10)$$

(Note that this is true by definition regardless of whether P_k has converged.)

We can plug in Equation 6 to get:

$$A_k = P_k^T (A P_k) = P_k^T P_{k+1} R_{k+1} \quad (11)$$

Let us define the orthogonal matrix $Q_{k+1} = P_k^T P_{k+1}$ using the fact that the product of two orthogonal matrices is also orthogonal (proof: $(AB)^{-1} = B^{-1} A^{-1} = B^T A^T = (AB)^T$). Therefore, A_k itself can now be written as a QR factorization:

$$A_k = Q_{k+1} R_{k+1} \quad (12)$$

Now consider A_{k+1} and attempt to form a recurrence relation:

$$\begin{aligned}
A_{k+1} &= P_{k+1}^T A P_{k+1} \\
&= P_{k+1}^T P_k A_k P_k^T P_{k+1} \\
&= Q_{k+1}^T A_k Q_{k+1} \\
&= Q_{k+1}^T Q_{k+1} R_{k+1} Q_{k+1} \\
&= R_{k+1} Q_{k+1}
\end{aligned} \tag{13}$$

Putting everything together, we have:

$$\begin{aligned}
A_k &= Q_{k+1} R_{k+1} \\
R_{k+1} Q_{k+1} &= A_{k+1}
\end{aligned} \tag{14}$$

Equation 14 is the essence of the QR algorithm:

1. Start with $A_0 = A$.
2. Find the QR factorization $A_k = Q_{k+1} R_{k+1}$.
3. Reverse the order to find $A_{k+1} = R_{k+1} Q_{k+1}$. Go to step 2.

If the algorithm converges, we have $Q_{k+1} = P_k^T P_{k+1} \approx P_k^T P_k = \mathbb{1}_n$. Therefore $A_k \approx R_k$, and we can read off the diagonal entries of A_k to find the eigenvalues by Theorem 2.

2.3 Example

Consider

$$\begin{aligned}
A &= \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix} \\
\det(A - \lambda \mathbb{1}_2) &= (2 - \lambda)^2 - 1 = 0 \\
&\Rightarrow \lambda_1 = 3, \lambda_2 = 1
\end{aligned} \tag{15}$$

Now let's run the QR algorithm. Let $A_0 = A$. Gram-Schmidt gives:

$$\begin{aligned}
\mathbf{q}_1 &= \frac{1}{\sqrt{5}} \begin{bmatrix} 2 \\ 1 \end{bmatrix} & Q_1 &= \frac{1}{\sqrt{5}} \begin{bmatrix} 2 & -1 \\ 1 & 2 \end{bmatrix} \\
\mathbf{v}_2 &= \begin{bmatrix} 1 \\ 2 \end{bmatrix} - \frac{1}{5} \begin{bmatrix} 2 \\ 1 \end{bmatrix} \begin{bmatrix} 2 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \end{bmatrix} & R_1 &= Q_1^{-1} A = \frac{1}{\sqrt{5}} \begin{bmatrix} 5 & 4 \\ 0 & 3 \end{bmatrix} \\
&= \frac{1}{5} \begin{bmatrix} -3 \\ 6 \end{bmatrix} & A_1 &= R_1 Q_1 \\
& & &= \frac{1}{\sqrt{5}} \begin{bmatrix} 5 & 4 \\ 0 & 3 \end{bmatrix} \frac{1}{\sqrt{5}} \begin{bmatrix} 2 & -1 \\ 1 & 2 \end{bmatrix} \\
\mathbf{q}_2 &= \frac{\mathbf{v}_2}{|\mathbf{v}_2|} = \frac{1}{\sqrt{5}} \begin{bmatrix} -1 \\ 2 \end{bmatrix} & &= \frac{1}{5} \begin{bmatrix} 14 & 3 \\ 3 & 6 \end{bmatrix} \\
& & & \lambda_1 \approx \frac{14}{5} = 2.8 \\
& & & \lambda_2 \approx \frac{6}{5} = 1.2
\end{aligned} \tag{16}$$

A second iteration gives:

$$\begin{aligned}
Q_2 &= \frac{1}{\sqrt{205}} \begin{bmatrix} 14 & -3 \\ 3 & 14 \end{bmatrix} \\
R_2 &= \frac{1}{\sqrt{205}} \begin{bmatrix} 41 & 12 \\ 0 & 15 \end{bmatrix} \\
A_2 &= R_2 Q_2 \\
&= \frac{1}{41} \begin{bmatrix} 122 & 9 \\ 9 & 42 \end{bmatrix} \\
\lambda_1 &\approx \frac{122}{41} \approx 2.976 \\
\lambda_2 &\approx \frac{42}{41} \approx 1.024
\end{aligned} \tag{18}$$

Computer simulation using Python gives the following eigenvalues for further iterations:

k	0	1	2	3	4	5	6
λ_1	2.00000000	2.80000000	2.97560976	2.99726027	2.99969521	2.99996613	2.99999624
λ_2	2.00000000	1.20000000	1.02439024	1.00273973	1.00030479	1.00003387	1.00000376

As we can see, the two eigenvalues converge fairly quickly to their precise values 3 and 1.

Improving the QR algorithm

The QR algorithm can be improved in many ways, including but not limited to:

- Reducing the matrix to upper Hessenberg form (almost triangular) before running the algorithm. This reduces the computational complexity of the algorithm.
- Running the algorithm on a shifted matrix ($A - s\mathbf{1}$) to maximize the convergence rate.

A thorough discussion of these improvements could be an interesting idea for future projects.

References

1. Jim Lambers, Advanced Topics in Numerical Linear Algebra, <https://web.stanford.edu/class/cme335/lecture4>
2. David Bindle, Matrix Computations, <https://www.cs.cornell.edu/~bindel/class/cs6210-f12/notes/lec24.pdf>
3. David Bindle, Matrix Computations, <https://www.cs.cornell.edu/~bindel/class/cs6210-f12/notes/lec25.pdf>
4. David Bindle, Matrix Computations, <https://www.cs.cornell.edu/~bindel/class/cs6210-f12/notes/lec26.pdf>
5. Gilbert Strang, Computing Eigenvalues and Singular Values, <https://ocw.mit.edu/courses/18-065-matrix-methods-in-data-analysis-signal-processing-and-machine-learning-spring-2018/resources/lecture-12-computing-eigenvalues-and-singular-values/>
6. Martijn Anthonissen, QR algorithm for computing eigenvalues, <https://www.youtube.com/watch?v=BfGAmw9qKsM>
7. Wikipedia, QR algorithm, https://en.wikipedia.org/wiki/QR_algorithm
8. Wikipedia, Schur decomposition, https://en.wikipedia.org/wiki/Schur_decomposition